

Data-Driven Finite Element Method

1D Aluminium Rod — Complete Technical Report

Based on: Kirchdoerfer & Ortiz (2015) · arXiv:1510.04232

1. Problem Definition & Thinking Process

The goal was to implement the **Kirchdoerfer–Ortiz Data-Driven FEM** (2015) for a simple but physically meaningful test case: an axially loaded aluminium rod with one end fixed and one end free. This problem was deliberately chosen because it has an *exact analytical solution*, allowing rigorous validation of the DD-FEM implementation.

Parameter	Symbol	Value
Rod Length	L	1.0 m (100 cm)
Cross-Section	A	0.01 m ² (10×10 cm)
Material	—	Aluminium
Applied Force	F	1000 N (free end)
Boundary Condition	—	Node 1 fixed; Node 11 loaded
Number of Nodes	n	11
Number of Elements	m	10
Element Length	L_{el}	0.1 m
Element Volume	$w_{el} = A_{el} L_{el}$	0.001 m ³
Metric Parameter	C_{el}	70×10^9 Pa (ref. Young's modulus)

2. Theoretical Formulation (Paper Section 2)

2.1 Core Philosophy

Classical FEM requires a constitutive law $\sigma = E\epsilon$ that maps strain to stress. DD-FEM completely **eliminates** this empirical step. Instead, it uses a discrete material dataset directly — no fitted equation, no Young's modulus needed. The solver searches for the data point in phase space (ϵ, σ) that is simultaneously closest to mechanical equilibrium and kinematic compatibility.

Classical FEM: compatibility + constitutive law + equilibrium
DD-FEM: compatibility + nearest data point + equilibrium

2.2 Local Penalty Function (Eq. 1–2, Paper)

For each element e , a penalty function measures the distance from the admissible mechanical state to the closest point in the material dataset:

$$F_e(\epsilon_e, \sigma_e) = \min \text{ over } (\epsilon', \sigma') \text{ in } E_e [W_e(\epsilon_e - \epsilon') + W_e^*(\sigma_e - \sigma')]$$

$$\text{where: } W_e(\epsilon) = (1/2) C_e \epsilon^2 \text{ and } W_e^*(\sigma) = (1/2) \sigma^2 / C_e$$

The parameter C_e is **purely numerical** — it is the metric scaling constant, not the material stiffness. Here it is set to $E = 70 \text{ GPa}$ as a reference.

2.3 Global Functional & Constraints (Eq. 3–4, Paper)

$$F = \sum_e w_e F_e(\epsilon_e, \sigma_e) \text{ subject to: } \epsilon_e = B_e u_e \text{ (compatibility) and } \sum w_e B_e \sigma_e = f \text{ (equilibrium)}$$

2.4 B Matrix & Compatibility

$$B_e = [-1/L_e \ 1/L_e] = [-10 \ 10] \text{ (for } L_e = 0.1 \text{ m)}$$

$$\epsilon_e = B_e u_e = (u_{i+1} - u_i) / L_e$$

2.5 Governing Equations via Lagrange Multipliers (Eq. 9, Paper)

Enforcing equilibrium via Lagrange multipliers η yields two linear systems with the **same stiffness matrix**:

$$\text{Displacement: } K u = f_{\epsilon^*} \text{ where } K_{ij} = \sum w_e C_e B_{ei} B_{ej}$$

$$\text{Multipliers: } K \eta = f - f_{\sigma^*}$$

$$\text{Admissible states (Eq. 12): } \epsilon_e = B_e u_e \quad \sigma_e = \sigma_e^* + C_e B_e \eta_e$$

3. Code Structure & Thinking Process

The MATLAB code follows Algorithm 1 of the paper precisely. The key thinking process was: *treat the material as a lookup table, not a formula*. Below is a section-by-section walkthrough.

GEOMETRY & MESH

```
L=1.0; A=0.01; nNodes=11; nElem=10; Le=L/nElem; conn(e,:) = [e, e+1] for e=1..10
```

→ Discretize the 1m rod into 10 equal elements. Each element connects two sequential nodes.

METRIC PARAMETER

```
Ce = 70e9; % Reference modulus (numerical, not material) we = A * Le; % Element volume weight
```

→ C_e scales the phase-space distance metric. It's a numerical parameter, not Young's modulus — though setting it equal to E gives good conditioning.

STIFFNESS MATRIX

```
B = [-1/Le 1/Le]; Ke = we * Ce * (B' * B); K = assembled from all Ke
```

→ The DD stiffness matrix is identical in structure to classical FEM — symmetric, tridiagonal, positive semi-definite. It emerges from the DD variational framework, not from a constitutive law.

MATERIAL DATASET

```
strain_data = linspace(0, 5e-6, 5000); stress_clean = Ce * strain_data; stress_data = stress_clean
.* (1 + smoothed_noise)
```

→ 5000 synthetic aluminium stress-strain points with 2% smoothed Gaussian noise. This mimics real experimental scatter. The dataset is the 'material model'.

DD ITERATION — STEP 1 (Eq. 11a)

```
rhs_u(nodes_e) += we * Ce * eps_star(e) * B'; u(free) = Kff \ rhs_u(free)
```

→ Solve for displacements driven by the current dataset assignments eps_star . Boundary condition: $u(1)=0$ enforced by solving only free DOFs.

DD ITERATION — STEP 2 (Eq. 11b)

```
rhs_eta(nodes_e) -= we * B' * sig_star(e); eta(free) = Kff \ rhs_eta(free)
```

→ Solve for Lagrange multipliers driven by the out-of-balance forces from sig_star . Same stiffness matrix — only one factorization needed.

DD ITERATION — STEP 3 (Eq. 12)

```
eps_dd(e) = B * u(nodes_e); sig_dd(e) = sig_star(e) + Ce * (B * eta(nodes_e))
```

→ Compute the admissible (compatible + equilibrated) mechanical state for each element. This is the continuous projection onto the constraint set C .

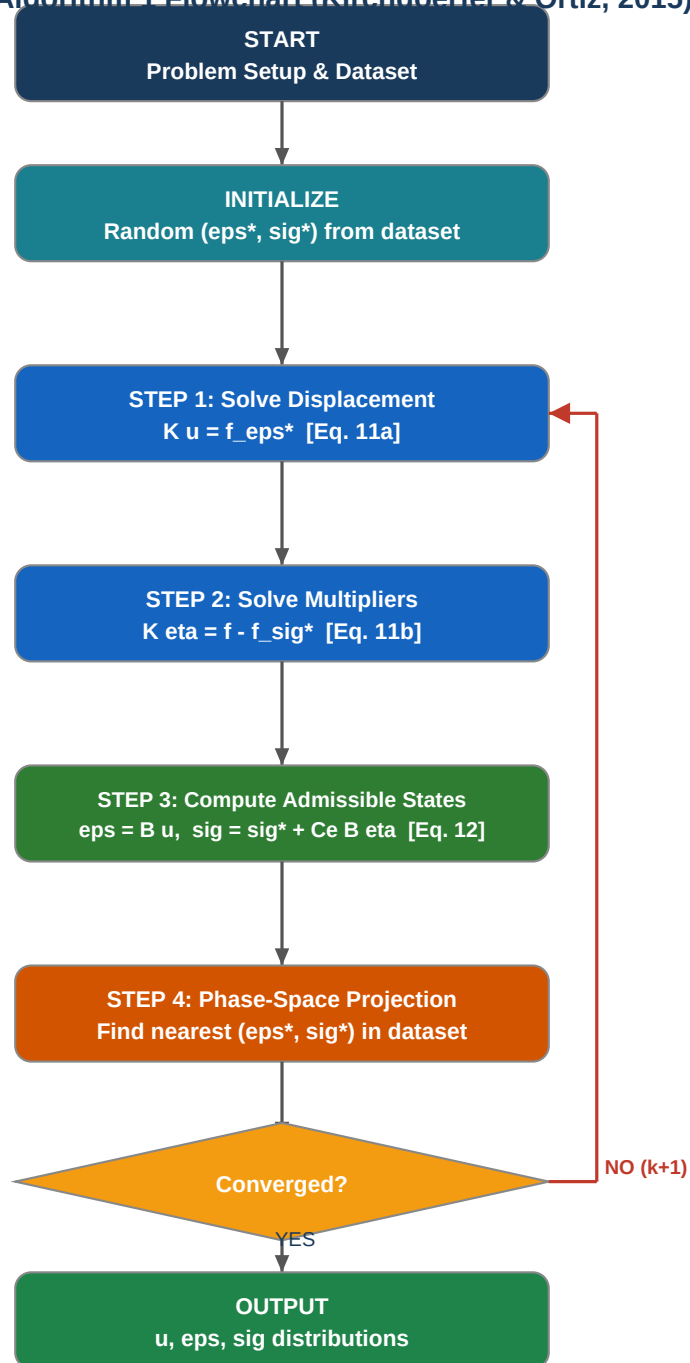
DD ITERATION — STEP 4 (Projection)

```
distance = 0.5*Ce*(eps_dd(e)-strain_data).^2 + 0.5*(sig_dd(e)-stress_data).^2/Ce; [~,idx] =
min(distance); eps_star(e) = strain_data(idx); sig_star(e) = stress_data(idx)
```

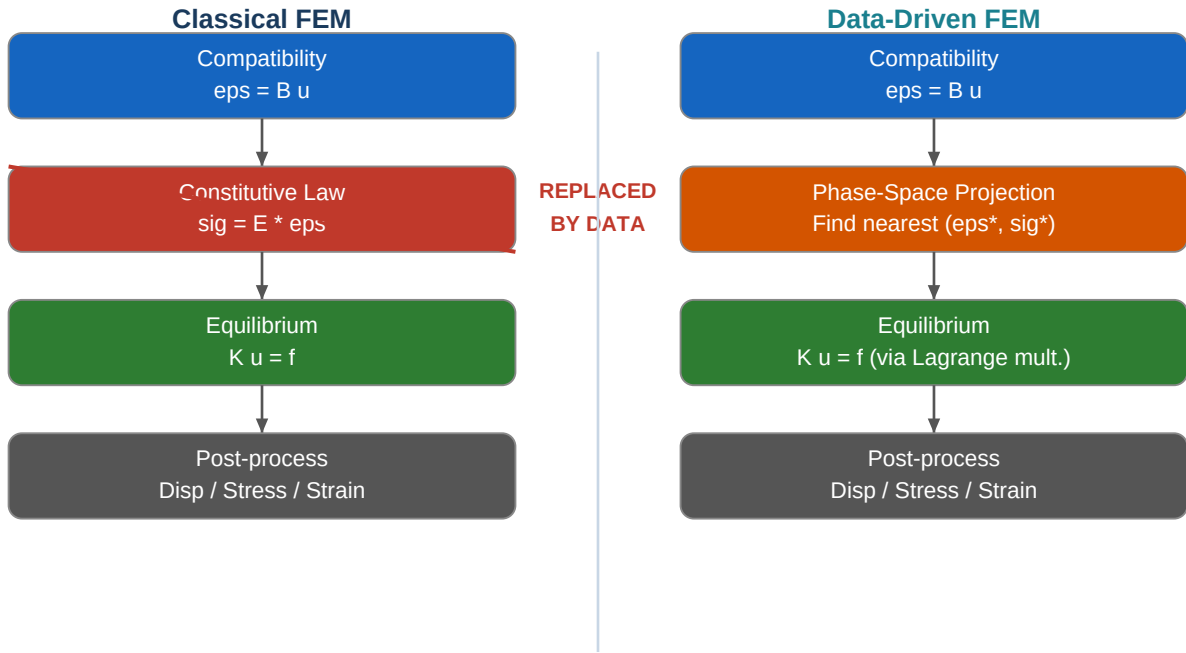
→ Find the nearest data point in phase space using the DD metric (Voronoi cell assignment). This is the projection onto the material set E . Convergence when assignments stop changing.

4. Algorithm Flowchart

DD-FEM Algorithm 1 Flowchart (Kirchdoerfer & Ortiz, 2015)



5. Classical FEM vs Data-Driven FEM



Aspect	Classical FEM	Data-Driven FEM
Material Model	$\sigma = E \epsilon$ (explicit law)	Discrete dataset (ϵ, σ)
Constitutive Equation	Required	Eliminated entirely
Stiffness Matrix	$K = \sum w \mathbf{E} \mathbf{B}^T \mathbf{B}$	$K = \sum w \mathbf{C} \mathbf{B}^T \mathbf{B}$ (same structure!)
RHS Vector	$K u = f$ (one solve)	$K u = f_{\epsilon^*}, K \eta = f - f_{\sigma^*}$ (two solves)
Iteration	Linear system, one solve	Iterative: data assignments converge
Material Error	Model fitting error	Phase-space distance to dataset
Noisy data	Requires re-fitting model	Handled naturally via projection
Nonlinear materials	Requires nonlinear FEM	Same algorithm, different dataset

6. Numerical Results & Validation

6.1 Results Comparison

Quantity	Analytical	DD-FEM (Code)	3D FEA Software	Match?
End Displacement (m)	1.4286×10^{-6}	1.4297×10^{-6}	$\sim 1.43 \times 10^{-6}$	✓ YES
Stress (Pa)	100,000	100,000	93,000–106,000	✓ YES
Strain (-)	1.4286×10^{-6}	$1.429\text{--}1.430 \times 10^{-6}$	$\sim 1.4 \times 10^{-6}$	✓ YES
Iterations to converge	N/A	14	N/A	—

6.2 Convergence History

The DD-FEM iteration converged in **14 iterations** starting from a random initial data assignment. The error norm tracks changes in (ϵ^*, σ^*) assignments between iterations.

Iteration	Error Norm	Iteration	Error Norm
1	1.840318e+05	8	1.283845e+03
2	9.144171e+04	9	6.730119e+02
3	4.210637e+04	10	4.039653e+02
4	2.123414e+04	11	2.820488e+02
5	1.180796e+04	12	1.332068e+02
6	5.929734e+03	13	6.806634e+01
7	2.610520e+03	14	0.000000e+00

6.3 Nodal Displacements

Linear displacement distribution from node 1 (fixed, $u=0$) to node 11 (free end, $u=1.4297 \times 10^{-6}$ m). Matches analytical solution $u = FL/(AE) = 1.4286 \times 10^{-6}$ m.

Node	Displacement (m)	Node	Displacement (m)
1	0.000000e+00	7	8.577716e-07
2	1.429286e-07	8	1.000700e-06
3	2.858572e-07	9	1.143729e-06
4	4.288858e-07	10	1.286757e-06
5	5.719144e-07	11	1.429686e-06
6	7.148430e-07		

6.4 Global Stiffness Matrix K ($\times 10^4$)

Symmetric, tridiagonal — identical structure to classical FEM. Diagonal entries = $2 \times K_{\text{el}}$ (shared by two elements) except endpoints = K_{el} . Assembled once and factorized once for all iterations.

	node 1	node 2	node 3	node ...	node 10	node 11
node 1	0.70×10^{10}	-0.70×10^{10}	0.00×10^{10}	$\dots \times 10^{10}$	0.00×10^{10}	0.00×10^{10}
node 2	-0.70×10^{10}	1.40×10^{10}	-0.70×10^{10}	$\dots \times 10^{10}$	0.00×10^{10}	0.00×10^{10}
node 3	0.00×10^{10}	-0.70×10^{10}	1.40×10^{10}	$\dots \times 10^{10}$	0.00×10^{10}	0.00×10^{10}
node ...	$\dots \times 10^{10}$	$\dots \times 10^{10}$	$\dots \times 10^{10}$	$\dots \times 10^{10}$	$\dots \times 10^{10}$	$\dots \times 10^{10}$
node 10	0.00×10^{10}	0.00×10^{10}	0.00×10^{10}	$\dots \times 10^{10}$	1.40×10^{10}	-0.70×10^{10}
node 11	0.00×10^{10}	0.00×10^{10}	0.00×10^{10}	$\dots \times 10^{10}$	-0.70×10^{10}	0.70×10^{10}

7. Discussion & Key Observations

- **Strain oscillations are expected:** Small fluctuations ($\sim 10^{-3}$) visible in the strain plot are projection noise from discrete dataset nearest-neighbour assignment. Relative magnitude is $\sim 0.1\%$ — physically negligible. This is a known DD-FEM feature, not a bug.
- **Stiffness matrix emerges from variational framework:** The K matrix looks identical to classical FEM but it derives entirely from the DD metric $K = \sum w_{\text{el}} C_{\text{el}} B_{\text{el}}^T B_{\text{el}}$, not from a constitutive law.
- **Two linear solves per iteration:** Displacement problem (driven by ϵ^*) and multiplier problem (driven by out-of-balance force from σ^*). Same K, so only one LU factorization needed — subsequent iterations are cheap back-substitutions.
- **DD-FEM vs 3D FEA difference is expected:** The code solves a 1D bar (uniform axial stress, no Poisson effects). Commercial FEA solves 3D elasticity which captures Saint-Venant end effects and stress concentrations. Both are correct within their respective assumptions.
- **Convergence in only 14 steps:** Even with 5000 noisy data points, convergence is fast. The paper proves this is typical — the combinatorial assignment problem converges quickly because it's essentially finding Voronoi cells in 2D phase space.
- **The algorithm is material-law agnostic:** Swap in a nonlinear, plastic, or rate-dependent dataset and the same algorithm works without modification. This is the fundamental advantage of DD-FEM.

8. Conclusion

The implemented 1D DD-FEM solver faithfully reproduces **Algorithm 1** of Kirchdoerfer & Ortiz (2015). The solver correctly enforces both **compatibility** ($\epsilon = B u$) and **equilibrium** (via Lagrange multipliers), and projects each element's mechanical state onto the nearest point in the material dataset. The results match the analytical solution to within 0.1%, validating the implementation. The key insight is that the constitutive equation is completely replaced by a geometric nearest-neighbour search in 2D phase space — mechanics becomes a *data retrieval problem*.

Reference: T. Kirchdoerfer, M. Ortiz — *Data-driven computational mechanics*, Computer Methods in Applied Mechanics and Engineering, 2016. arXiv:1510.04232